



High-Level Takeaways

- Managing when processes run
 - " memory
 -
- } OS making hardware shareable
"fairness"

Takeaway 1: Abstraction is the core technique.

- file descriptor: where writing to / from
- virtual address: where in RAM the data lives
- process: hides that the CPU is shared.

Takeaway 2: Isolation requires hardware cooperation.

- Vurt. Mem: MMU
- Kernel/user mode separation

Pattern: OS sets the ds/register,
but hardware enforces.

Takeaway 3: Tradeoffs!

- Monitors to avoid race conditions, but cost is performance.
- Space vs. Time: - page tables: flat vs. multi-level.
- Performance vs. Simplicity:
- Responsiveness vs. Throughput:
Process scheduling.
- Transparency vs. efficiency: `mmap()` vs. `read()`/`write()`.

Takeaway 4: The Slow path is the dangerous path.

- TLB:
- Mutex: "popular" / contested vs. available.
- Cache hit vs miss.

Know the fast path, + protect it.

Takeaway 5: Concurrency bugs are qualitatively different than regular bugs

Week 1:

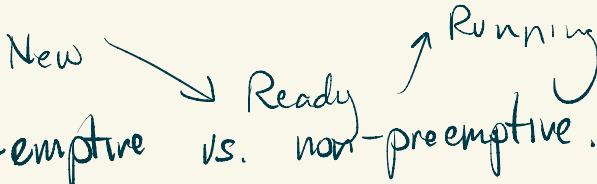
- Resource Management
- Hardware Abstraction.

Processes:

- instance of running program
 - logical control flow
 - private address space
- "illusion" of multiprocessing
- States: Running, Stopped, terminated, getpid(), fork(), exit(), wait(), exec()
- shell

Scheduling:

- Process state diagram



- Pre-emptive vs. non-preemptive.

- Context switching

- Metrics:

- turnaround time
- response time
- throughput
- fairness

- FCFS / FIFO

- SJF

- STCF

- RR

- Priority scheduling

- MLFQ

- Lottery, stride

Concurrency / Threads

- Program Structure

Responsiveness

Performance

- Race conditions

- Problems:

concurrent reads, writes, updates.

- Threads vs. Processes:

- Threads share memory
- own stack/registers!

thread_create()

- exit()

- join()

"Sequential consistency".

Synchronization:

- shared resource $\Rightarrow$ PROTECT!

- Critical sections
- Locks / Mutexes
- Condition variables
- Monitors
- Dahlin's 6 Rules:

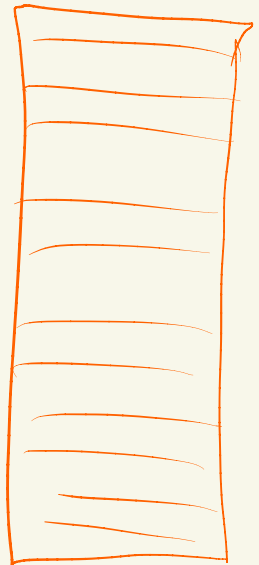
- Always do things the same way.
 - Always use monitors.
 - Always hold the lock when using cond. var.
 - Acquire at start, release before return
 - Always use while, not if.
 - Never sleep(). (Almost/probably).
- Deadlock
 - Live lock
 - Starvation
 - Performance vs. complexity: coarse vs fine-grained locking.

Virtual Memory

- $VA \rightarrow PA$.
- Page:
- $VPN + offset \rightarrow PPN + offset$
- Page table: $VPN \rightarrow PPN \leftarrow 1/process$
- Multilevel page table
- VPN/PPN bit calculations, page table walks.
- x86-64 addresses
- Page table entries (PTE)
- TLB
- Page faults

* Mapping multiple VAs to one PAs: do they have to be read-only?

- Programs clash w/o it.
- Memory is a nightmare to manage.
(need address @ compile time).
- You can't have more memory than we physically have.



Page Fault:

HW tries to map VA, fails,

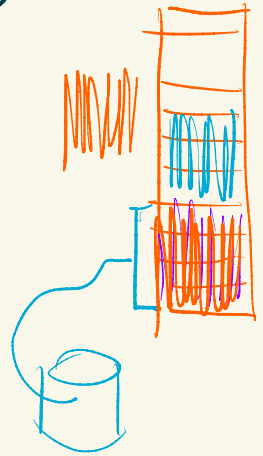
① Protection violation.

② Page is not present.

⇒ bring page into memory,
update PTE,
restart the instruction.

-
- Lazy allocation.
 - Page swapped to disk.
 - mmap.

- The OS does not save a process's entire memory when it gets deprioritized for the CPU.



I/O:

- mmap vs read()
- polling vs. interrupts.
- DMA: Direct Memory Access.
- Device drivers
- Sync vs. Async I/O.
 - blocking vs. non-blocking
- Disks,
 - HDD.
 - SSD

File Systems

- Map names → disk block.
- persistence
- naming
- mapping